Centro Universitário Álvares Penteado
Projeto Interdisciplinar: Ciência de Dados

2ª Entrega PI: Aplicação de Design de Software e Diagramas UML no Projeto Interdisciplinar **Objetivo:** Aplicar design de software e UML em projeto real, analisando divergências, mudanças e melhorias necessárias para a entrega do PI.		
Nomes: Julia Pereira Godinho | RA: 26028796 Mariana Almeida Nascimento | RA: 22011051 Rafaela Florêncio Morais | RA: 25027460	Profº Luis Pires	Turma: CCOMP 4

DESIGN DE SOFTWARE E DIAGRAMAS UML NO PROJETO INTERDISCIPLINAR

Título do Projeto

CTI Analytics: Plataforma Analítica de Planejamento Financeiro, desenvolvida em parceria com a empresa CTI Global, especializada em gestão corporativa, planejamento e controle financeiro.

Documento de referência

Este documento tem como base a especificação "CTI – Plataforma Analítica de Planejamento Financeiro", fornecida pelos professores orientadores e pela parceira CTI Global, e dá continuidade à 1ª Entrega de Engenharia de Software (Engenharia de Requisitos e Métodos Ágeis).

1. Contextualização

Na 1ª Entrega, transformamos a especificação do Projeto Interdisciplinar em requisitos funcionais, requisitos não funcionais e regras de negócio, e organizou o trabalho com o método Scrumban no GitHub Projects. Nesta 2ª Entrega, o foco passa a ser o design de

software: como a solução CTI Analytics foi estruturada para atender a esses requisitos, quais decisões de arquitetura foram tomadas e como elas são representadas por diagramas UML.

O objetivo deste documento é aplicar design de software e UML em um projeto real, apresentando a arquitetura do painel analítico, ao menos dois diagramas UML (neste documento, três), os resultados aplicados ao projeto determinando as observações reflexivas do grupo, analisando as divergências entre o que foi planejado na 1ª Entrega e o que foi efetivamente construído, bem como as melhorias necessárias para a entrega final.

2. Design de Software e Arquitetura do Sistema

2.1. Visão geral

A solução foi projetada com uma arquitetura modular em três camadas, Dados, Processamento e Apresentação, seguindo o requisito não funcional (separação entre dados, processamento e apresentação). A camada de Dados organiza as informações em raw, staging e processed, preservando a base original da CTI de forma imutável. A camada de Processamento reúne o tratamento e a validação, o cálculo de KPIs, a análise de sensibilidade, o ranking financeiro e as análises estatísticas (descritiva e regressão). A camada de Apresentação concentra o dashboard web desenvolvido em Streamlite com gráficos em Plotly.

2.2. Organização do repositório

A estrutura de diretórios do repositório reflete as camadas descritas acima:

- **data/raw/**: base original da CTI, preservada sem alterações;
- **data/staging/**: dados tratados e padronizados (limpeza, encoding, formatos numéricos e contábeis);
- **data/processed/**: resultados consolidados (KPIs, ranking, sensibilidade e estatísticas) consumidos pelo dashboard;
- **notebooks/**: notebooks (Colab/Jupyter) de coleta, exploração, tratamento e análise estatística;
- **src/**: código modular em Python — tratamento e validação, cálculo de KPIs, sensibilidade, ranking, estatística e aplicação do dashboard;
- **docs/**: catálogo de fontes, dicionário de dados e dicionário de KPIs;
- **ES e ML/**: documentos de Engenharia de Software (Entrega 1 e Entrega 2).

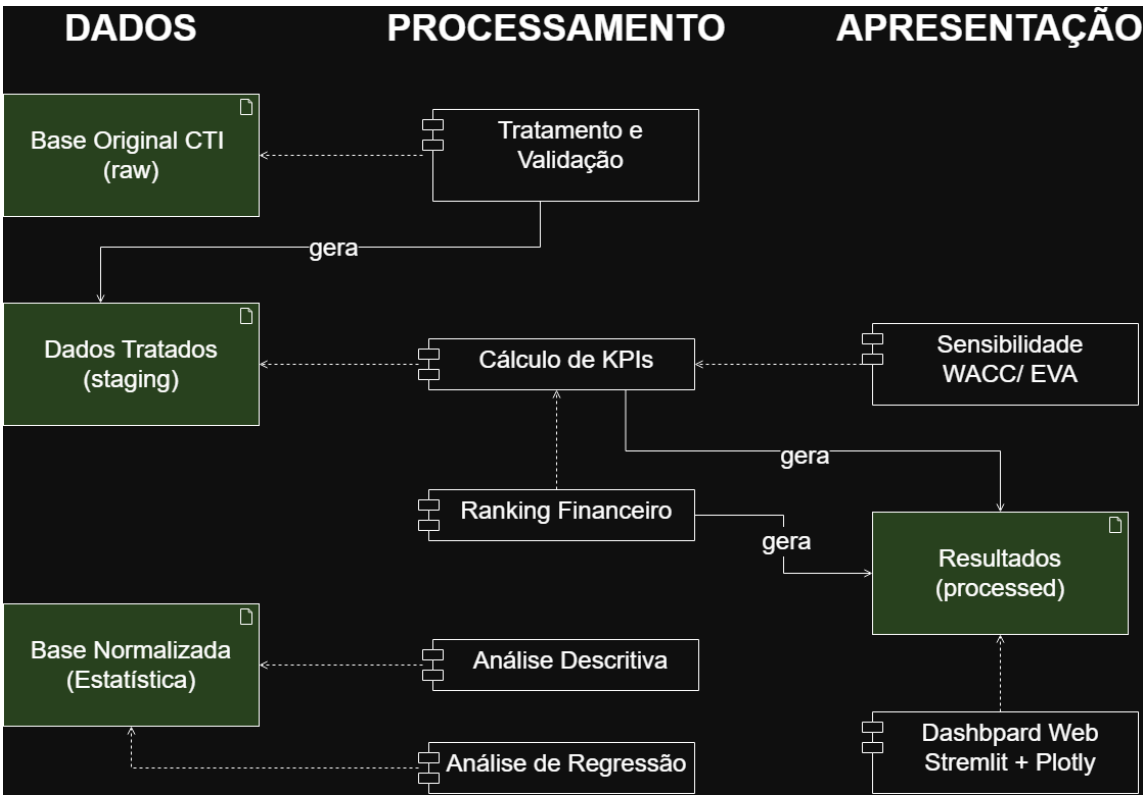
2.3. Decisões de design

- **Dados brutos imutáveis:** a base original da CTI nunca é sobrescrita; todo tratamento gera uma nova camada (staging), garantindo rastreabilidade e reprodutibilidade.
- **Regras financeiras isoladas da interface:** fórmulas de KPIs, EVA/WACC e ranking ficam em módulos próprios, de modo que ajustes contábeis não exijam alterações no layout do dashboard.
- **Resultados pré-processados:** o dashboard lê apenas a camada processed por meio de um carregador único, o que melhora o desempenho de filtragem e evita lógica duplicada na interface.
- **Base normalizada separada:** a análise descritiva e a regressão operam sobre uma base normalizada própria, sem alterar os dados usados no cálculo dos KPIs financeiros.
- **Sensibilidade parametrizada:** a análise de sensibilidade reutiliza os KPIs existentes e recalcula o EVA para diferentes valores de WACC, atendendo ao requisito de simulação de cenários (RF11).

3. Diagramas UML

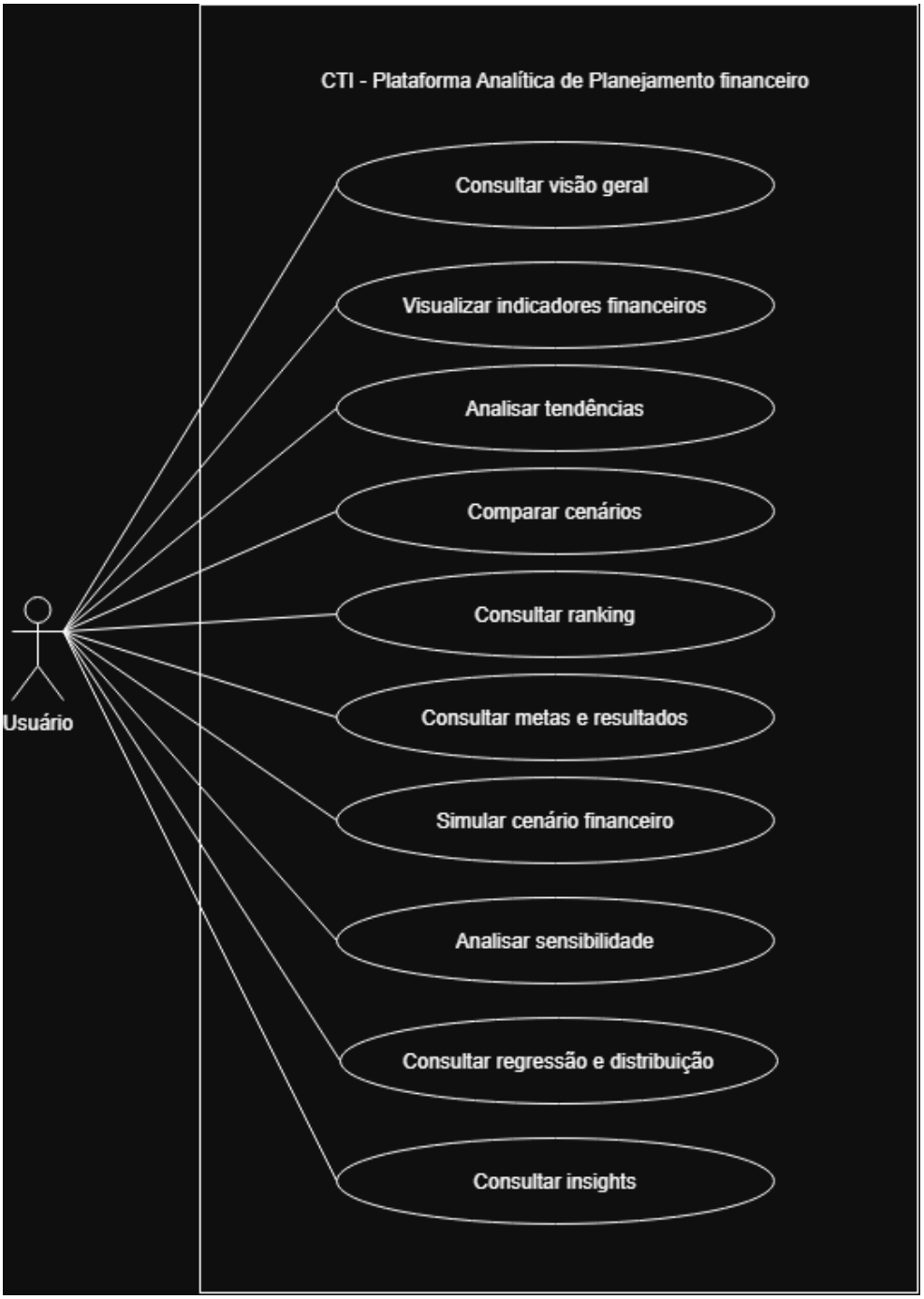
3.1. Diagrama de Componentes

O diagrama de componentes mostra como a arquitetura do sistema foi dividida em partes modulares para separar bem as responsabilidades. Dá para ver claramente a divisão entre as camadas de dados (que cuidam do armazenamento das bases *raw*, *staging* e normalizada), a parte de processamento e inteligência (onde rodam as validações, os cálculos de KPIs, as estatísticas e as regressões) e, na ponta, a camada de apresentação com o dashboard em Streamlit e Plotly, que pega tudo o que foi processado e entrega a interface pronta para o usuário interagir.



3.2. Diagrama de Casos de Uso

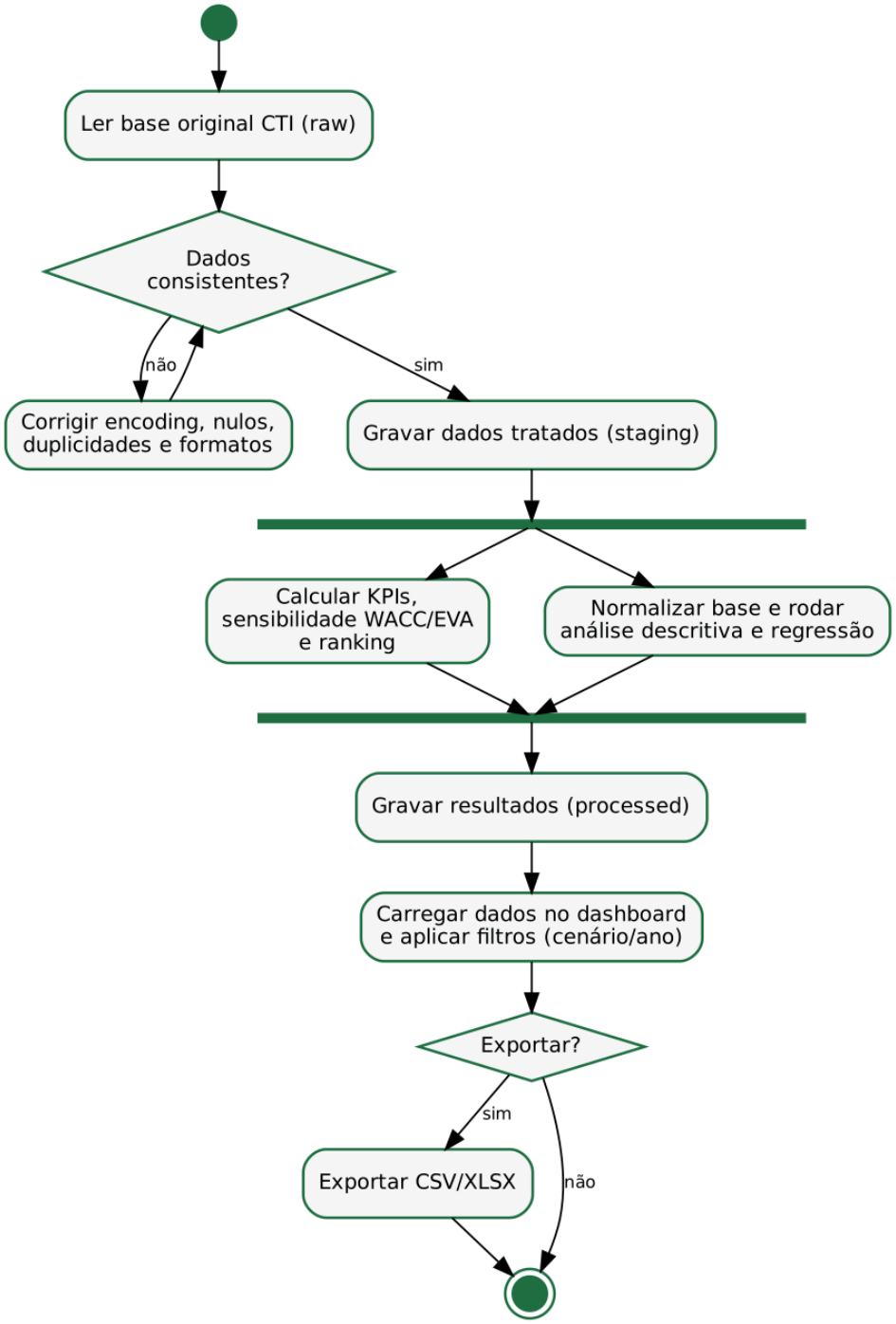
O diagrama de caso de uso mostra as interações entre o usuário e o sistema, detalhando todas as funcionalidades que a plataforma disponibiliza. O ator principal (o usuário) conecta-se diretamente ao escopo da aplicação para realizar ações como consultar a visão geral, visualizar indicadores financeiros, analisar tendências, comparar cenários, verificar o ranking, conferir metas e resultados, simular cenários futuros, fazer análises de sensibilidade, consultar regressões e distribuições estatísticas, além de explorar os insights gerados pela ferramenta.



3.5. Diagrama de Atividades

O diagrama de atividades detalha o passo a passo de como o sistema executa as tarefas desde o começo até a entrega final. O fluxo começa lendo a base de dados bruta e já passa por um teste para ver se os dados estão consistentes: se tiver erro, o sistema roda uma rotina para limpar códigos quebrados, nulos e duplicados antes de salvar a base tratada. Logo depois, o processo se divide para rodar em paralelo os cálculos financeiros/rankings e as análises

estatísticas, cujos resultados são salvos e carregados no dashboard com filtros de ano e cenário, dando ainda a liberdade para o usuário exportar os dados em CSV ou XLSX se quiser.



6. Conclusão

A modelagem em UML nos obrigou a olhar a solução como um conjunto de responsabilidades, e não apenas como um conjunto de scripts. Ao revisar o diagrama de componentes elaborado antes da conclusão do dashboard, percebemos lacunas que não eram

visíveis no início, como a falta de um componente responsável por gerar a base normalizada e a ausência de ligação entre as análises estatísticas e o painel.

A separação entre dados, processamento e apresentação mostrou-se uma boa decisão, pois permitiu dividir o trabalho entre os integrantes e alterar as regras de cálculo dos KPIs sem refazer a interface. Por outro lado, constatamos que parte dos requisitos da Entrega 1 foi priorizada de forma diferente da planejada, o que reforça a importância de revisar o backlog e a documentação a cada etapa.

Como aprendizado, o grupo concluiu que manter diagramas e requisitos atualizados em paralelo ao código reduz retrabalho e facilita a explicação do projeto na apresentação final.